

# RAG Chatbot

## A Complete Code Explanation for Beginners

*This document explains, line by line, a Python program that builds a **Retrieval-Augmented Generation (RAG) chatbot**. The chatbot reads a PDF file, stores its content in a smart database, and uses an AI model to answer questions about it. No prior coding experience needed — every concept is explained simply.*

### Contents

|    |                            |   |
|----|----------------------------|---|
| 1  | The Big Picture            | 2 |
| 2  | Architecture Diagram       | 2 |
| 3  | Imports — gathering tools  | 3 |
| 4  | Configuration & secrets    | 3 |
| 5  | The AI brain (LLM)         | 4 |
| 6  | The memory database        | 4 |
| 7  | The embedding function     | 4 |
| 8  | Helper functions           | 5 |
| 9  | Reading & indexing the PDF | 5 |
| 10 | Retrieving relevant chunks | 6 |
| 11 | Getting the AI's answer    | 6 |
| 12 | Running the chatbot        | 7 |
| 13 | Glossary                   | 7 |

# 1. The Big Picture

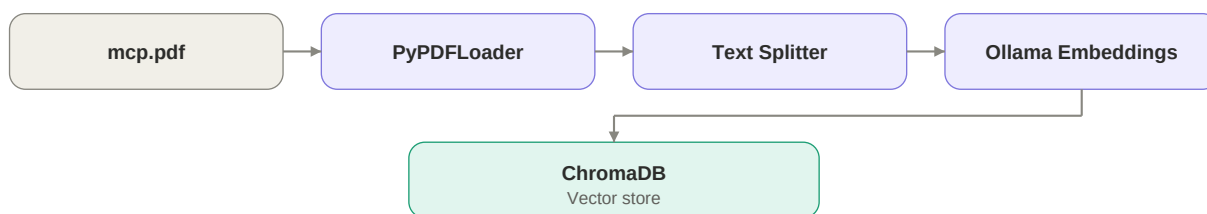
This script builds a **RAG chatbot** — a program that can read a PDF document and then answer questions about it intelligently. RAG stands for **Retrieval-Augmented Generation**, which means: instead of the AI guessing from memory, it first *retrieves* the relevant parts of the document, then *generates* an accurate answer based on those parts.

*Think of it like an open-book exam. The AI is allowed to look up the relevant pages of a textbook before answering each question — so the answers are far more accurate than if it had to rely on memory alone.*

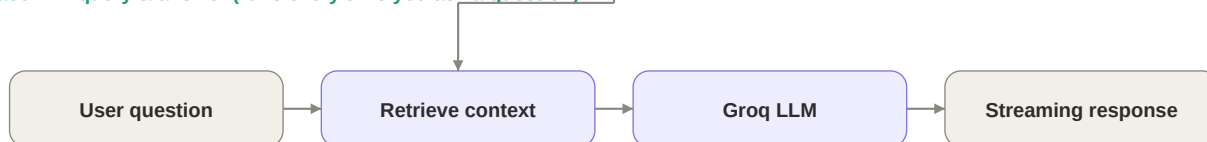
## 2. Architecture Diagram

The diagram below shows the two phases of the system. **Phase 1** (top, purple) runs once when the script starts and indexes the PDF. **Phase 2** (bottom) runs every time you ask a question — it finds the relevant chunks and sends them to the AI.

Phase 1 — document indexing (runs once at startup)



Phase 2 — query & answer (runs every time you ask a question)



  Processing step      Vector store      User-facing

*ChromaDB sits in the centre as the bridge: Phase 1 fills it with knowledge, Phase 2 queries it for relevant information.*

### 3. Imports — Gathering Tools

The first lines of every Python program import the tools (called *libraries*) it needs. Think of this like getting out all your cooking ingredients before you start making a meal.

```
from dotenv import load_dotenv
from os import getenv
import os, re, ollama, chromadb
from langchain_core.messages import HumanMessage, SystemMessage
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_groq import ChatGroq
from system_prompt_simple import system_prompt
```

|               |                                                                                                                         |
|---------------|-------------------------------------------------------------------------------------------------------------------------|
| load_dotenv   | Reads secret keys from a hidden <code>.env</code> file, like a safe for passwords.                                      |
| os / re       | <code>os</code> lets Python talk to files & folders. <code>re</code> finds patterns inside text (like a search filter). |
| ollama        | Converts text into lists of numbers called <i>embeddings</i> — the AI's way of understanding meaning.                   |
| chromadb      | A special database that stores those number-fingerprints and can search them super fast.                                |
| langchain_*   | A toolkit for loading PDFs, splitting them into pieces, and talking to AI models.                                       |
| ChatGroq      | The connector to Groq's fast AI service — this is the brain that writes answers.                                        |
| system_prompt | A pre-written rulebook that tells the AI how to behave (tone, format, what to focus on).                                |

### 4. Configuration & Secret Keys

Next the script reads configuration values and secret API keys from the environment.

```
load_dotenv()
MODEL = getenv('GROQ_MODEL', 'llama-3.1-8b-instant')
MBED_MODEL = getenv('EMBED_MODEL', 'embeddinggemma:latest')
GROQ_API_KEY = getenv('GROQ_API_KEY')
DOCUMENT_PATH = os.path.join(os.path.dirname(__file__), 'mcp.pdf')

if not GROQ_API_KEY:
    raise RuntimeError('GROQ_API_KEY not found...')
```

|                    |                                                                                                                                  |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------|
| load_dotenv()      | Opens the hidden <code>.env</code> file and loads all the secrets into memory.                                                   |
| getenv('KEY', def) | Reads a setting by name. If it's missing, falls back to the default value given.                                                 |
| DOCUMENT_PATH      | Builds the full file path to the PDF that will be read and indexed.                                                              |
| raise RuntimeError | A <i>guard</i> : if the API key is missing, the program stops immediately with a clear error message — like a bouncer at a door. |

## 5. Creating the AI Brain

```
llm = ChatGroq(model=MODEL, temperature=0)
```

|                            |                                                                                                                                        |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <code>ChatGroq(...)</code> | Creates a connection to Groq's AI service. Think of this as hiring a very smart assistant.                                             |
| <code>model=MODEL</code>   | Chooses which AI model to use — like picking which expert to hire.                                                                     |
| <code>temperature=0</code> | Sets creativity to zero: the AI gives consistent, factual answers. A higher value (e.g. 0.9) makes it more creative and unpredictable. |

## 6. Setting Up the Memory Database

```
collection = chromadb.Client().create_collection(name='knowledge_base')
all_chunks: list[str] = []
```

|                                     |                                                                                                        |
|-------------------------------------|--------------------------------------------------------------------------------------------------------|
| <code>chromadb.Client()</code>      | Creates a ChromaDB database in RAM (temporary, resets when the program stops).                         |
| <code>create_collection(...)</code> | Creates a named filing cabinet inside the database where chunks will be stored.                        |
| <code>all_chunks: list[str]</code>  | A plain Python list (like a notepad) that also keeps a copy of every text chunk for keyword searching. |

## 7. The Embedding Function

```
def get_embedding(text):
    return ollama.embeddings(model=EMBED_MODEL, prompt=text)['embedding']
```

This function converts any piece of text into a **list of numbers** called an *embedding*. Sentences with similar meanings get similar numbers — like GPS coordinates for meaning. This lets the computer find related sentences by comparing numbers, which is extremely fast.

*Example: 'The cat sat on the mat' and 'A feline rested on the rug' would get very similar embeddings, even though no words are shared.*

## 8. Helper Functions

Three small utility functions clean up data before it is used.

### `_chunk_to_text(content)`

The AI can return its response in different formats (plain text, a list of pieces, or other structures). This function normalises all of them into a single clean string so the rest of the code doesn't have to worry about the format.

### `_prepare_retrieval_query(user_input)`

Takes the user's raw question and removes blank lines and extra whitespace. This makes sure the search uses a clean, single-line query.

### `_keyword_tokens(text)`

```
stop = {'who', 'what', 'when', 'the', 'and', 'for', 'was', 'are', ...}
return [token for token in re.findall(r'[a-z0-9]+', text.lower())
        if token not in stop]
```

Extracts meaningful keywords from a question by removing common 'stop words' (words like 'the', 'and', 'how' that appear everywhere and carry no useful search signal). What remains are the important terms like *'transformer'* or *'attention'*.

## 9. Reading & Indexing the PDF

This is the core of Phase 1. The function reads the PDF, cuts it into chunks, embeds each chunk, and stores everything in ChromaDB.

```
def build_vector_store():
    docs = PyPDFLoader(DOCUMENT_PATH).load()
    chunks = RecursiveCharacterTextSplitter(
        chunk_size=1000, chunk_overlap=200
    ).split_documents(docs)
    for i, chunk in enumerate(chunks):
        all_chunks.append(chunk.page_content)
        collection.add(
            ids=[f'chunk_{i}'],
            embeddings=[get_embedding(chunk.page_content)],
            documents=[chunk.page_content]
        )
```

|                                             |                                                                                                                                                                                                       |
|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>PyPDFLoader(...).load()</code>        | Opens and reads the entire PDF file into memory, page by page.                                                                                                                                        |
| <code>RecursiveCharacterTextSplitter</code> | Cuts the full text into overlapping chunks of ~1000 characters. The 200-character overlap means the boundary between chunks always has context from both sides — no information is lost at the edges. |
| <code>enumerate(chunks)</code>              | Loops through every chunk, giving each one an index number (0, 1, 2, ...).                                                                                                                            |
| <code>collection.add(...)</code>            | Saves each chunk plus its embedding fingerprint into ChromaDB under a unique ID.                                                                                                                      |

# 10. Retrieving Relevant Chunks

When you ask a question, this function finds the PDF chunks most likely to contain the answer — using **two search strategies** in parallel.

```
def retrieve_context(question, top_k=15):
    query = _prepare_retrieval_query(question)

    # Strategy 1 – Semantic search
    results = collection.query(
        query_embeddings=[get_embedding(query)],
        n_results=top_k)
    semantic = (results['documents'] or [[]])[0]

    # Strategy 2 – Keyword search
    keywords = _keyword_tokens(query)
    scored = sorted(all_chunks,
        key=lambda c: sum(1 for kw in keywords if kw in c.lower()),
        reverse=True)
    keyword_hits = [c for c in scored[:10]
        if c not in semantic_set
        and any(kw in c.lower() for kw in keywords)]

    return '\n\n'.join(semantic + keyword_hits)
```

| Strategy | How it works                                                         | Best for                                      |
|----------|----------------------------------------------------------------------|-----------------------------------------------|
| Semantic | Converts question to embedding, finds chunks with similar embeddings | Meaning-based matches (synonyms, paraphrases) |
| Keyword  | Counts how many question words appear literally in each chunk        | Exact matches (names, numbers, codes)         |

Using both strategies together gives broader, more reliable coverage — like using a map AND road signs to navigate.

# 11. Getting the AI's Answer

```
def get_streaming_response(user_input):
    context = retrieve_context(user_input)
    messages = [
        SystemMessage(content=system_prompt),
        HumanMessage(content=f'Context:\n{context}\n\nQuestion: {user_input}'),
    ]
    yield ('thinking', 'Retrieved relevant chunks. Asking Groq...')
    for chunk in llm.stream(messages):
        text = _chunk_to_text(chunk.content)
        if text:
            yield ('response', text)
```

|                       |                                                                               |
|-----------------------|-------------------------------------------------------------------------------|
| retrieve_context(...) | Finds the most relevant PDF sections for this question.                       |
| SystemMessage         | The AI's rulebook — tells it how to behave and what role to play.             |
| HumanMessage          | Your question, bundled together with the retrieved PDF context as background. |

|                                   |                                                                                                                                        |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <code>llm.stream(messages)</code> | Sends everything to Groq and receives the reply word-by-word as it's generated, just like text appearing live in ChatGPT.              |
| <code>yield</code>                | Instead of waiting for the whole answer, this hands each small piece back immediately as it arrives — that's called <i>streaming</i> . |

## 12. Running the Chatbot

```
build_vector_store()          # runs immediately — index the PDF

if __name__ == '__main__':
    while True:
        user_input = input('You: ')
        if user_input.lower() == 'quit':
            break
        for kind, text in get_streaming_response(user_input):
            print(text, end='', flush=True)
        print('\n')
```

|                                             |                                                                                                          |
|---------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <code>build_vector_store()</code>           | Called immediately at startup — reads and indexes the PDF before any questions are asked.                |
| <code>if __name__ == '__main__':</code>     | Python's way of saying 'only run this block if this file is run directly, not imported by another file'. |
| <code>while True:</code>                    | An infinite loop — keeps the chatbot alive, asking for the next question.                                |
| <code>if ... == 'quit': break</code>        | The exit door: typing 'quit' stops the loop and ends the program.                                        |
| <code>print(..., end='', flush=True)</code> | Prints each streaming word without adding a newline, and flushes immediately so it appears live.         |

## 13. Glossary

|                     |                                                                                                        |
|---------------------|--------------------------------------------------------------------------------------------------------|
| <b>RAG</b>          | Retrieval-Augmented Generation — an AI technique that looks up relevant documents before answering.    |
| <b>Embedding</b>    | A list of numbers representing the meaning of a piece of text. Similar texts get similar numbers.      |
| <b>Vector store</b> | A database optimised for storing and searching embeddings by similarity.                               |
| <b>Chunk</b>        | A small piece of a larger document, typically a few hundred words.                                     |
| <b>LLM</b>          | Large Language Model — a powerful AI trained on huge amounts of text to generate human-like responses. |
| <b>Streaming</b>    | Sending the response word-by-word as it's generated, instead of waiting for the full answer.           |
| <b>API key</b>      | A secret password that proves you have permission to use an online service.                            |
| <b>Temperature</b>  | A setting (0–1) controlling how creative/random an AI's output is. 0 = consistent, 1 = creative.       |

---

*End of document — happy coding! ■*